



CUSTOMER SUPPORT TICKET SYSTEM

8-5-2026

Introduction

This document explains the project structure, components, how to set up development, and how to deploy using Docker, Docker Compose, and Kubernetes. The stack is intended to run on Linux (Docker Engine, Kind).

Content

0. Introduction.....	2
1. Product overview.....	4
2. Architecture.....	5
✓ 2.1 Microservices (HTTP APIs + MongoDB).....	5
✓ 2.2 How services communicate.....	5
✓ 2.3 Docker containerization (images, containers, files).....	6
3. Repository structure.....	8
4. Prerequisites.....	9
✓ 4.1 All modes.....	9
✓ 4.2 Docker Compose.....	9
✓ 4.3 Kubernetes (optional path).....	9
5. Development setup.....	10
✓ 5.1 Full stack with Docker Compose.....	10
✓ 5.2 Environment matrix (Compose).....	10
✓ 5.3 MongoDB Compass.....	10
6. Building container images manually.....	11
7. Deploying with Kubernetes (Kind).....	12
✓ 7.1 Build images.....	12
✓ 7.2 Load into Kind (cluster name cst).....	12
✓ 7.3 Apply an overlay.....	12
✓ 7.4 Verify.....	12
✓ 7.5 Production overlay notes.....	12
8. Testing.....	13
✓ 8.1 Frontend unit tests (Vitest via Angular).....	13
✓ 8.2 Smoke checks (Compose).....	13
9. Troubleshooting.....	14
10. Security reminders.....	15
11. Quick reference commands.....	16

1. Product overview

The system supports:

- ✓ **Customers:** submit tickets, track status, receive notifications from agents.
- ✓ **Agents:** manage tickets (assign, update), log responses, trigger customer notifications.
- ✓ **DevOps:** containerized microservices, multiple isolated environments on one host, and Kubernetes manifests for orchestration.

2. Architecture

2.1 Microservices (HTTP APIs + MongoDB):

Service	Role	Default port (in container)
Frontend	Angular SPA served by nginx; reverse-proxies API paths to backends	80 (mapped to host per env)
Auth Service	Registration, login, JWT	5001
Ticket Service	CRUD tickets, status, authorization via JWT/headers	5000
Support Service	Assignment and response workflow endpoints	5002
Notification Service	Persist and list notifications for customers	5003
MongoDB	Data store (separate logical databases per concern)	27017 (internal)

2.2 How services communicate

- **Browser** → frontend (nginx) on the published UI port (e.g. 8080).
- **Nginx** → backends over HTTP inside the Compose/Kubernetes network, using Docker/Kubernetes service DNS names (auth-service, ticket-service, etc.).
See `docker/nginx/default.conf`.
- **Backends** → MongoDB via `MONGODB_URI` (Mongo wire protocol on `mongo:27017` inside the network).

2.3 Docker containerization (images, containers, files)

This section summarizes **what runs in Docker** for grading and operations clarity.

⇒ **Services in one Compose stack (docker-compose.yml):**

#	Compose service name	Runs as container?	Image source
1	mongo	Yes	Pull: mongo:7.0
2	auth-service	Yes	Build → cst-auth-service:<tag>
3	ticket-service	Yes	Build → cst-ticket-service:<tag>
4	notification-service	Yes	Build → cst-notification-service:<tag>
5	support-service	Yes	Build → cst-support-service:<tag>
6	frontend	Yes	Build → cst-frontend:<tag>

- ✓ **Containers per full stack: 6** (one container per service above).
- ✓ **Images used by that stack: 6 distinct image references**
 - **1** third-party image: mongo:7.0
 - **5** application images built from this repo (each service name + digest differs).

The image tag suffix comes from `IMAGE_TAG_SUFFIX` in `env/*.env` (e.g. dev, test, staging, prod), so a running dev stack might use `cst-auth-service:dev`, etc.

⇒ **Custom Dockerfiles (build definitions)**

Dockerfile	Builds how many app images	Notes
docker/node-service.Dockerfile	4 images	Same multi-stage pattern; SERVICE_DIR selects auth-service, ticket-service, notification-service, or support-service
docker/frontend.Dockerfile	1 image	Multi-stage: Node builds Angular → nginx:1.26-alpine serves static assets + reverse proxy config from docker/nginx/default.conf.

So there are **2 Dockerfile definitions** producing **5 custom images** (plus pull **Mongo** as a pre-built image).

Counting **distinct image names** in one stack (excluding duplicate tags of the same logical app):

1. mongo:7.0
2. cst-auth-service:*
3. cst-ticket-service:*
4. cst-notification-service:*
5. cst-support-service:*
6. cst-frontend:*

That is **more than three**; replicas of the same image do not apply here (default replica count is **1** per service in Compose).

Multiple environments on one machine

If you start **all four** env files (development, testing, staging, production) with scripts/compose-up-all-parallels.sh:

- ✓ **Total running containers:** $4 \times 6 = 24$ (four isolated Compose projects, each with its own 6 containers).
- ✓ **Named volumes:** 4 Mongo data volumes (one per project; name includes COMPOSE_PROJECT_NAME and MONGO_VOLUME_NAME).

Each project has its **own** Docker network; containers in project A **do not** share a network with project B (only with the other five services in the same docker-compose project).

⇒ Published host ports (typical):

Only **frontend** and **mongo** publish ports to the host in Compose; backends are reached via the nginx gateway or internal DNS.

Exposure	Service	Purpose
FRONTEND_HOST_PORT → 80	frontend	Browser UI + API proxy paths
MONGO_HOST_PORT → 27017	mongo	MongoDB Compass / external tools

See env/*.env for the actual numbers (e.g. dev UI 8080, dev Mongo host port 27021).

⇒ Kubernetes (same logical images):

Kubernetes manifests under kubernetes/ reference the same application image **names** (cst-auth-service:latest, etc.). Build with scripts/k8s-build-images.sh, then load or push those images so the cluster can run **one Pod per service** (frontend can scale to multiple replicas in the base manifest).

3. Repository structure

```

Customer-Support-Ticket-System-main/
├── frontend/           # Angular 21 app
├── backend/
│   ├── auth-service/
│   ├── ticket-service/
│   ├── notification-service/
│   └── support-service/
├── docker/
│   ├── node-service.Dockerfile # Reusable Node (Alpine) image per microservice
│   ├── frontend.Dockerfile    # Multi-stage Angular build + nginx
│   └── nginx/default.conf     # API routing for containerized frontend
├── env/
│   ├── development.env
│   ├── testing.env
│   ├── staging.env
│   └── production.env        # Env-specific Compose variables (ports, DB names, secrets)
├── scripts/
│   ├── compose-up-stack.sh    # One stack: up -d --build
│   ├── compose-up-all-parallel.sh
│   ├── compose-env.sh         # docker-compose / docker compose wrapper + --env-file
│   ├── down-all-environments.sh
│   ├── k8s-build-images.sh
│   └── k8s-apply.sh           # kubectl apply -k overlay
├── kubernetes/
│   ├── base/                 # Kustomize base (deployments, services, mongo PVC)
│   └── overlays/
│       ├── development/
│       ├── testing/
│       ├── staging/
│       └── production/       # Includes sample Ingress
├── docker-compose.yml        # Single file; behaviour driven by env/*.env
├── .dockerignore
└── DOCUMENTATION.md          # This file

```

Legacy / reference only (commented, do not use)

- ✓ docker-compose.dev.yml, docker-compose.test.yml, docker-compose.prod.yml — historical commented drafts; use instead docker-compose.yml # Single file; behaviour driven by env/*.env
- ✓ k8s/*.yaml — old flat manifests; **use kubernetes/** with Kustomize.
- ✓ scripts/up-all-environments.sh — disabled; use compose-up-all-parallel.sh.

4. Prerequisites

4.1 All modes

- ✓ **Linux** host with **Docker** (daemon running).
- ✓ User in docker group **or** use sudo with Docker (less convenient).
- ✓ **Node.js** (e.g. 22.x) and **npm** for local (non-container) frontend dev.

4.2 Docker Compose

- ✓ Prefer **Docker Compose v2** (docker compose).
- ✓ If only v1 is installed, **docker-compose** is supported by the helper scripts.

4.3 Kubernetes (optional path)

- ✓ kubectl (e.g. sudo snap install kubectl --classic).
- ✓ A cluster: **Kind**, **Minikube**, **k3d**, or a cloud cluster.
- ✓ Cluster must be able to pull or load the images built locally (kind load docker-image, etc.).

5. Development setup

5.1 Full stack with Docker Compose:

From the repository root:

```
# Single environment (example: development)
bash scripts/compose-up-stack.sh env/development.env
```

UI (development): <http://localhost:8080/> (see `FRONTEND_HOST_PORT` in each `env/*.env`).

Bring all environments up concurrently (different ports and DB volumes):

```
bash scripts/compose-up-all-parallel.sh
```

Stop all:

```
bash scripts/down-all-environments.sh
```

Inspect one stack (works with Compose v1 or v2):

```
bash scripts/compose-env.sh env/testing.env ps
bash scripts/compose-env.sh env/development.env config
```

5.2 Environment matrix (Compose):

Environment	UI port	Mongo host port	Compose project name
development	8080	27021	cst-development
testing	8081	27018	cst-testing
staging	8090	27019	cst-staging
production	8082	27020	cst-production

Each environment uses different Mongo database names (`MONGO_DB_*` in the env file), different `JWT_SECRET`, and different image tag suffixes.

5.3 MongoDB Compass:

Connect to the host port for the stack you started, e.g.:

- ✓ Development: `mongodb://127.0.0.1:27021`

Data is split across databases, for example in dev:

- ✓ `cst_auth_dev` → users
- ✓ `cst_ticket_dev` → tickets
- ✓ `cst_notify_dev` → notifications

6. Building container images manually

Images use multi-stage builds for smaller production images.

```
# All backend tags + frontend  
bash scripts/k8s-build-images.sh
```



Produces tags such as:

- ✓ cst-auth-service:latest
- ✓ cst-ticket-service:latest
- ✓ cst-notification-service:latest
- ✓ cst-support-service:latest
- ✓ cst-frontend:latest

7. Deploying with Kubernetes (Kind)

7.1 Build images:

```
bash scripts/k8s-build-images.sh
```

7.2 Load into Kind (cluster name cst):

```
kind load docker-image cst-auth-service:latest --name cst
kind load docker-image cst-ticket-service:latest --name cst
kind load docker-image cst-notification-service:latest --name cst
kind load docker-image cst-support-service:latest --name cst
kind load docker-image cst-frontend:latest --name cst
```

7.3 Apply an overlay:

```
bash scripts/k8s-apply.sh testing
```

Overlays live under `kubernetes/overlays/{development,testing,staging,production}`.

7.4 Verify:

```
kubectl get pods -n customer-support-testing
kubectl get svc -n customer-support-testing
```

Access UI via port-forward:

```
kubectl port-forward -n customer-support-testing svc/frontend 9081:80
```

Open <http://localhost:9081>.

7.5 Production overlay notes:

`kubernetes/overlays/production` includes an Ingress example (`cst-production.local.example`).
Adjust host and `ingressClassName` / annotations for your cluster's ingress controller.

8. Testing

8.1 Frontend unit tests (Vitest via Angular):

```
cd frontend  
npx ng test --watch=false
```



8.2 Smoke checks (Compose):

With a stack up:

```
curl -sf http://localhost:8080/api/tickets # development UI port
```



9. Troubleshooting

Issue	What to try
unknown flag: --env-file for docker compose	Use docker-compose (v1) or bash scripts/compose-env.sh env/....env ...
Docker permission denied on socket	sudo usermod -aG docker \$USER then log out/in
docker-compose ContainerConfig KeyError on recreate	Remove stale containers or docker-compose down then up
Mongo port already in use	Change MONGO_HOST_PORT in the matching env/*.env
Kubernetes pods ImagePullBackOff	Load/push images; check image names match manifests
Compass empty DB	Select correct database names (cst_*) not only ticketDB

10. Security reminders

non-production samples:

- ✓ Replace JWT_SECRET in every env/*.env before any real deployment.
- ✓ Mongo in these samples has **no auth**; enable authentication and network policies for production.
- ✓ The legacy k8s/ tree is **disabled**; prefer **kubernetes/** overlays and sealed secrets / external secret managers in real prod.

11. Quick reference commands

```
# One env up
bash scripts/compose-up-stack.sh env/development.env

# All envs up
bash scripts/compose-up-all-parallels.sh

# All envs down
bash scripts/down-all-environments.sh

# K8s apply testing overlay
bash scripts/k8s-apply.sh testing
```

For questions about Angular-only development, see [frontend/README.md](#).